

 Engine_1_trade...
16.64 KB

For better performance, previous chat history has been compacted.

1. The OMS/accounting layer is not exchange-true. You are tracking per-strategy "positions," but Binance one-way Futures is symbol-netted unless explicitly managed otherwise. Your code cancels all symbol orders, uses `closePosition=true`, treats `orderId` like a position ticket, and reconciles PnL from `userTrades?limit=1`. This can create naked windows, wrong exits, and completely wrong PnL attribution.

2. **The GTX → market fallback loop is structurally adverse-selected.** You get maker fills mainly when short-term flow moves against the order, and you chase with market orders when the signal is right. Backtest likely prices entries near bar open/close/mid and does not model this conditional cost.
3. **Your live exit/risk logic does not match the backtest.** The backtest activates trailing after `TP_MULT * ATR = 5R`; `Engine_1.py` activates around `2 * ATR`. The min-stop floor widens SL without rescaling TP/trailing. `STOP_MARKET` slippage/price-protect behavior is not modeled. This alone can turn a high-WR backtest into live chop.

Also: the trade log itself proves reconciliation corruption. Example: a long SUI trade from 0.656 to 0.684 is logged as negative PnL. That is impossible from price movement and indicates wrong fill/PnL assignment.

0. Smoking guns from your current code/log

A. PnL ledger is untrustworthy



From the JSON:

- SUIUSD `LONG`: entry 0.656, exit 0.684, units 9234.5, logged PnL -16.76.
 - Gross should be about $9234.5 * (0.684 - 0.656) = +258.57$ USD before fees.
- ADAUSD `LONG`: entry 0.1739, exit 0.1824, units 22910, logged PnL -24.20.
 - Gross should be about +194.74 USD.
- TRXUSD `LONG`: entry 0.33233, exit 0.33285, units 18806, logged PnL -28.53.
 - Gross should be about +9.78 USD, before fees.

This points directly to this pattern:

```
res = self._request("GET", "/fapi/v1/userTrades",  
                    params={"symbol": symbol, "limit": 1}, signed=True)
```

Copy

`get_last_fill(symbol)` is assigning the **most recent fill for the symbol** to whichever local trade happens to be stale. With multiple strategies on the same symbol, this is guaranteed to misattribute fills.

This contaminates:

- `current_capital`
- drawdown governor
- cooldowns
- strategy loss filters
- retraining labels/callbacks
- your ability to diagnose actual slippage



Before optimizing execution, fix the ledger.

B. You are running multiple local trades on one Binance net position

Examples:

- SUIUSD has `S4_Mean_Reversion` and `S5_Vol_Breakout` live near the same time.
- ADAUSD has multiple strategy trades close together.
- TRXUSD repeatedly re-enters under `S4`.

In Binance one-way Futures, there is one net positionAmt per symbol. Your local system treats these as independent positions, each with independent SL/TP. But your broker uses:

```
"closePosition": "true"
```

Copy

and before attaching a new SL:

```
self._cancel_all_orders(binance_symbol)
```

Copy

That means a new trade can cancel the old trade's protective stop for the same symbol, then attach a single close-all stop for the entire net position.

This is a major structural mismatch.

C. Backtest trailing != live trailing

Backtest in `six_strategy_engine.py`:

```
td = atr * TP_MULT          # TP_MULT = 5.0  
...  
if (bp - entry) >= td:  
    ns = bp - trd
```



Copy

So trailing activates after `5 * ATR`.

Live in `Engine_1.py`:

```
trail_activate_at = 2.0 * entry_atr
```

Copy

That is not the same strategy. You are training/testing labels for one exit policy and executing another.

Example: SUIUSD `S5` entry 0.656, ATR 0.001378.

- Backtest trail activation: $0.656 + 5 * ATR = 0.66289$
- Live trail activation: $0.656 + 2 * ATR = 0.65876$

Live starts trailing far earlier, so many potential backtest winners become small wins or scratch/fee losses.

D. Min stop floor changes SL but not the full trade geometry

In `trigger_entry()`:

```
if stop_dist < min_stop_dist:  
    stop_dist = min_stop_dist  
    if direction == 1:  
        sl = entry_price - stop_dist  
    else:  
        sl = entry_price + stop_dist
```



Copy

But TP remains the old `5 * ATR` target. For low ATR symbols like TRX/DOGE, this destroys the intended R multiple.

Example style:

- Original backtest: SL = 1 ATR, TP activation = 5 ATR

- Live after floor: SL maybe 3 ATR, TP still 5 ATR.
- Expected SR trade becomes maybe 1.5R or less before fees/slippage.

If a stop floor is needed, either reject the trade or rescale the whole bracket/trailing logic.

1. Root cause analysis: top 3 live execution leaks

Leak 1 — OMS/state model mismatch with Binance net positions

What is happening

Your local model:

```
trade_id -> independent strategy position
```

Copy

Binance one-way Futures model:

```
symbol -> one net positionAmt
```

Copy

Your code then compounds the mismatch:

```
_cancel_all_orders(symbol)
```



Copy

before placing a new SL. This cancels protective orders belonging to other local trades on the same symbol.

Then SL placement uses:

```
closePosition=true
```

Copy

so a single triggered stop closes the whole symbol position, not one strategy's allocation.

Reconciliation uses invented "tickets":

```
PositionObj(ticket)
```

Copy

but Binance Futures does not have MT5-style position tickets. orderId is not a position id.

Why it destroys edge

This causes:

- old stops being canceled by new entries
- one local trade's stop closing another trade's position
- wrong trade getting marked closed
- false BROKER_SYNC losses
- emergency closes against already-flat or unrelated positions
- local strategy state diverging from real exchange state
- PnL attribution becoming garbage



Code-level symptoms

```
def _cancel_all_orders(self, binance_symbol: str):  
    self._request("DELETE", "/fapi/v1/allOpenOrders", params={"symbol": binance_symbol},  
signed=True)  
    ...  
    DELETE open algo orders too
```

Copy

This should not run before every new SL attachment if multiple local trades can exist on the same symbol.

Also:

```
def get_last_fill(self, symbol: str):  
    params={"symbol": symbol, "limit": 1}
```

Copy

This is not trade-specific. It is symbol-specific and race-prone.

Required fix

Choose one of these models:

Option A — simplest and safest

Enforce:

```
max 1 exchange-managed trade per symbol
```

Copy

No multiple strategies on the same symbol simultaneously.

In `trigger_entry()`:



```
if any(t["symbol"] == symbol for t in self.active_trades.values()):  
    return
```

Copy

This alone will remove a huge class of failures.

Option B — symbol-level position manager

Allow multiple strategies internally, but exchange execution is managed as one aggregate symbol position:

```
symbol_position:  
    net_qty  
    avg_entry  
    aggregate_stop  
    synthetic_strategy_allocations[]
```

Copy

There is one real Binance position, one real protective stop, and internal PnL allocation by strategy.

Option C — separate subaccounts

If strategies truly require independent SL/TP and independent inventory, use separate Binance subaccounts or isolated execution accounts. Hedge mode gives only one LONG and one SHORT bucket per symbol, not arbitrary strategy-level independent positions.

Leak 2 — GTX maker fallback loop creates adverse selection

Current behavior

You place passive GTX:

LIMIT + GTX



Copy

For long:

```
anchor = live_bid  
limit_price = anchor - offset
```

Copy

So you are bidding at/behind the bid.

If not filled within a few seconds, you cancel and send market.

Why this is structurally bad

For a long signal:

- If your passive bid fills, it is usually because sellers hit you.
 - That is negative short-term flow.
 - You capture half-spread only in the cases where the market is moving into you.
- If your passive bid does not fill, it often means price moved up.
 - That is when your signal was probably right.
 - You then chase with a market buy at a worse price.

So live conditional cost is:

```
Expected cost =  
P(passive fill) * (maker fee/rebate - half spread + adverse markout)  
+ P(no fill) * (taker fee + half spread + chase drift + impact)
```

Copy



Backtest probably assumes something like:

```
entry = next_bar_open  
or entry = signal_close  
or entry = mid_price
```

Copy

That misses the conditional fill penalty.

Additional bugs in current implementation

Partial fills are ignored

In the GTX loop:

```
if fetched_order.get('status') == 'FILLED':  
    filled = True
```

Copy

You ignore:

PARTIALLY_FILLED

Copy

Then on timeout you cancel and submit a market order for the full slice. If the passive order partially filled, you can overfill.

Correct flow must be:

```
place passive  
poll/order-update  
if partial fill:  
    record executedQty and cumQuote  
cancel remainder  
fetch final order  
market/IOC only remaining quantity
```



Copy

Cancel/fill race is not handled

This is dangerous:

```
self._cancel_limit_order(symbol, order_id)
```

Copy

You do not fetch final order state after cancel. On exchanges, an order can fill while cancel is in flight. You need:

```
cancel request  
fetch order  
account for executedQty  
only trade remaining quantity
```

Copy

Average price is wrong for sliced fills

You use only the last entry_result:

```
cum_quote = float(entry_result.get("cumQuote", 0.0))  
exec_qty = float(entry_result.get("executedQty", 0.0))  
avg_price = cum_quote / exec_qty
```

Copy

But for multiple slices, VWAP must be:



```
avg = sum(order_i.cumQuote) / sum(order_i.executedQty)
```

Copy

Current SL/TP distance can be wrong after multi-slice execution.

Required fix

Do not use GTX followed by market chase unless your backtest explicitly models that conditional behavior.

Use one of these:

Mode 1 — pure marketable limit IOC with slippage collar

For long:

```
limit_price = current_ask * (1 + max_slippage_bps)  
timeInForce = IOC
```

Copy

For short:

```
limit_price = current_bid * (1 - max_slippage_bps)
```

Copy

```
timeInForce = IOC
```

This behaves like a market order but caps worst acceptable fill.

Mode 2 — passive-or-cancel, no chase

If you want maker economics:

```
post passive
if not filled within X ms, cancel signal
do not market chase
```

That is a different strategy but avoids paying worst-case taker cost exactly when price is running away.

Mode 3 — adaptive execution

For slower alpha:

```
quote passively only when spread/depth favorable
cancel if markout deteriorates
cross only if alpha edge >> spread + taker + expected slippage
```

But that needs a real execution model.

Leak 3 — backtest/live exit policy and stop execution mismatch

STOP_MARKET slippage

Your backtest fills stops exactly at stop price:

```
if l[j] <= cs:
    ep = cs
```

Live STOP_MARKET does not fill at stop price. It triggers at stop, then becomes a market order.

For long stop:

```
bad slippage = (stop_trigger - sell_fill_vwap) / stop_trigger
```

For short stop:

```
bad slippage = (buy_fill_vwap - stop_trigger) / stop_trigger
```

In fast altcoin moves, this can be several bps to dozens of bps. On tight ATR strategies, that is fatal.

priceProtect=true can delay/reject stop triggers

You use:

```
"priceProtect": "true",
"workingType": "MARK_PRICE",
```

This can be sensible for manipulation protection, but it also means the actual triggering mechanics differ from a backtest using trade OHLC.

If mark price and contract price diverge during liquidation bursts, a price-protected stop may not trigger when your local price logic expects.

Live trail mismatch

Backtest:

```
trail activates at 5 ATR
trail distance = 0.8 ATR
```

Live:

```
trail activates at 2 ATR
trail distance = 1.0 ATR in Engine_1.py comments/code
```

This is not the same strategy.

TP behavior is inconsistent

binance_broker.execute_trade() computes TP but initially only places SL:

```
sl_res = self.place_algo_conditional(... STOP_MARKET ...)
```

There is no initial TP placement in that function.

modify_sltp() later tries to place TP, but your heartbeat likely never runs because:

```
if now - getattr(self, 'last_sl_heartbeat', now) > 60:
```

If last_sl_heartbeat does not exist, default is now, so the condition is always false. Unless another path sets it, this heartbeat never starts.

Trailing modifications can still call modify_sltp(), but TP attachment is inconsistent.

Required fix

Make the live exit engine exactly match the backtest, or regenerate labels/backtests for the live policy.

At minimum:

```
trail_activate_at = trade.get("Intended_tp_distance", 5.0 * entry_atr)
trail_dist = 0.8 * entry_atr
```

And if SL is widened by a min-stop floor:

- either reject the trade
- or rescale TP/trail off the widened stop distance and retrain/retest

2. Backtest vs live assumption matrix

Area	Backtest assumption	Current live reality	Failure mode	Fix
------	---------------------	----------------------	--------------	-----

Entry price	Next open / close / mid	GTX bid/ask then market fallback	Adverse selection + chase	Model execution cost or use IOC collar
Maker fill	Passive fill is "good"	Passive fill occurs conditionally when flow hits you	Negative markout	Log 1s/5s markout after maker fills
No-fill	Often ignored	No-fill causes market chase	Worst fills on strongest moves	Passive-or-cancel or taker-only with collar
Spread	Often zero or fixed	Dynamic per symbol/time	Tight ATR edge erased	Log decision spread bps
Fees	Static round-trip	Actual maker/taker tier, BNB discounts, rebates uncertain	Fee assumptions wrong	Use actual commissions from fills
Stop fill	Fill exactly at SL	STOP_MARKET trigger, then sweeps book	Loss > planned R	Measure stop slippage bps
TP fill	Exact TP/trailing logic	TP initially not placed; later modify may fail	Strategy mismatch	One canonical bracket policy
Trailing	5 ATR activation	2 ATR activation live	Premature exits	Match TP_MULT
SL floor	Not modeled	SL widened without TP rescale	R/R collapses	Reject or rescale and retest
Intra-bar path	OHLC deterministic	Tick path + queue + latency	Stop/entry ordering wrong	Tick/orderbook replay or calibrated slippage
Partial fills	Usually full fill	Partials ignored in code	Overfill, wrong VWAP	Account executedQty after cancel
Position model	Independent trades	Binance one-way net position	Stops cancel/close wrong trades	One trade per symbol or aggregate OMS
Order state	Synchronous fill	ACK/NEW/PARTIAL/EXPIRED/CANCELLED async	Ghost trades	User data WS state machine
Reconciliation	Trade-specific fills	userTrades limit=1 symbol fill	Impossible PnL	Fill ledger by client/order IDs
REST freshness	Instant state	Polling delay / transient failures	False flat BROKER_SYNC	User WS primary, REST snapshot fallback
Startup	Clean state	Existing positions/orders possible	Cancel protective stops	Reconcile before cancel
Funding	Often ignored	8h funding over 72h max hold	PnL drift	Include funding in backtest
Tick/lot rules	Continuous price/qty	Discrete tick/step/min notional	Rejections/rounding drift	Decimal exchange filters
Testnet	Representative	Testnet liquidity is not mainnet	Bad fill calibration	Calibrate on mainnet small size

3. Architectural blueprint

Phase 0: emergency stabilization

Do these before further live trading.

1. Enforce one active trade per symbol

Until the OMS is rewritten:

```
if any(t["symbol"] == symbol for t in self.active_trades.values()):
    log_live_event(f"{symbol} blocked: existing symbol position active", "RiskGov")
    return
```

This prevents multiple strategies from fighting over one Binance net position.

2. Stop canceling all symbol orders before placing SL

This is dangerous:

```
self._cancel_all_orders(binance_symbol)
```

Do not cancel old protective orders before new protection exists. Also do not cancel every algo order for the symbol unless you own it and have reconciled the position.

Use bot-owned client IDs:

```
E1_<strategy>_<symbol>_<trade_id>_SL
E1_<strategy>_<symbol>_<trade_id>_ENTRY
```

Cancel only orders with your prefix and only after replacement protection is confirmed.

3. Disable GTX fallback

Temporarily replace with:

```
LIMIT IOC with slippage collar
```

For buy:

```
price = ask * (1 + max_slip_bps / 10000)
```

For sell:

```
price = bid * (1 - max_slip_bps / 10000)
```

This avoids unbounded market slippage and avoids the passive-fill/chase asymmetry.

4. Fix live training parity

Replace:

```
trail_activate_at = 2.0 * entry_atr
```

Copy

with:

```
trail_activate_at = trade.get("intended_tp_dist", 5.0 * entry_atr)
trail_dist = 0.8 * entry_atr
```

Copy

If you intentionally want 2R trailing, then regenerate labels and retrain on the 2R trailing policy.

5. Reject min-stop-floor trades initially

Do not widen SL while leaving TP unchanged.

Instead:

```
if stop_dist < min_stop_dist:
    log_live_event(f"{symbol} rejected: stop below executable floor", "RiskGov")
    return
```

Copy

Afterward, build a separate backtest variant that includes stop floors.



6. Fix PnL attribution immediately

Delete/disable this for production PnL:

```
get_last_fill(symbol, limit=1)
```

Copy

You need fills linked by:

- clientId
- orderId
- trade timestamp window
- side
- reduce-only/entry classification

Never use "last symbol fill" as a trade close.

Phase 1: WebSocket-first exchange state

You need two WebSocket systems.

A. Market data stream

Maintain fresh local cache:

- @bookTicker or depth stream
- @depth5@100ms or similar for top depth
- @markPrice
- trades/aggTrades if needed

Each cache object must contain:



```
@dataclass
class BookTop:
    symbol: str
    bid: float
    ask: float
    bid_qty: float
    ask_qty: float
    event_ms: int
    recv_ns: int

    def age_ms(self) -> float:
        return (time.time_ns() - self.recv_ns) / 1e6
```

Copy

Execution should fail closed if:

```
book.age_ms() > 250-500ms
spread_bps > symbol_threshold
depth < order_size * depth_multiplier
```

Copy

Your current REST check:

```
ticker = GET /ticker/bookTicker
```

Copy

has no freshness guarantee and no local latency measurement.

B. User data stream

This is non-negotiable.



Use user data stream events for:

- order accepted
- order partially filled
- order filled
- canceled
- expired
- stop triggered
- account/position update

REST should be fallback only:

```
startup_snapshot
reconnect_recovery
periodic_audit
```

Copy

The primary state machine should be driven by exchange events, not polling.

Phase 2: proper OMS state machine

Use explicit states.

```
from enum import Enum
```

Copy

```
class TradeState(str, Enum):
    SIGNAL_RESERVED = "SIGNAL_RESERVED"
    ENTRY_SUBMITTED = "ENTRY_SUBMITTED"
    ENTRY_ACKED = "ENTRY_ACKED"
    ENTRY_PARTIAL = "ENTRY_PARTIAL"
    ENTRY_FILLED = "ENTRY_FILLED"
    PROTECTION_SUBMITTED = "PROTECTION_SUBMITTED"
    PROTECTED = "PROTECTED"
    LIVE = "LIVE"
    EXIT_SUBMITTED = "EXIT_SUBMITTED"
    CLOSED = "CLOSED"
    REJECTED = "REJECTED"
    RECOVERY_PENDING = "RECOVERY_PENDING"
    UNKNOWN = "UNKNOWN"
```

Important rule:

```
A trade is not LIVE until:
filled_qty > 0
and protective SL for filled_qty is confirmed active
```

If entry fills but SL placement fails:

```
state = RECOVERY_PENDING
send reduce-only emergency close
do not remove local trade until exchange confirms flat
```

Phase 3: bracket management

For one-position-per-symbol mode

Use one protective stop per symbol.

Prefer quantity-specific reduceOnly stops over closePosition=true if you are managing discrete quantities.

For initial SL:

```
STOP_MARKET
side = opposite
quantity = filled_qty
reduceOnly = true
workingType = MARK_PRICE or CONTRACT_PRICE, but match backtest/measurement
```

For TP:

- If strategy is trailing-only, do not place hard TP.
- If hard TP is desired, use reduce-only LIMIT TP; not TAKE_PROFIT_MARKET, unless you model TP slippage.

For aggregate symbol manager

Maintain:

```
symbol_position.qty
symbol_position.avg_entry
symbol_position.exchange_stop_id
symbol_position.synthetic_allocations[]
```

Then update the single exchange stop based on aggregate risk.

4. Code-level mitigation patterns

Pattern 1: exact exchange rounding with Decimal

Your `_round_step()` is unsafe for non-power-of-ten. Use Decimal.

```
from decimal import Decimal, ROUND_FLOOR, ROUND_CEILING, ROUND_HALF_UP

def round_to_step(value: float, step: float, mode: str = "nearest") -> float:
    v = Decimal(str(value))
    s = Decimal(str(step))
    if s <= 0:
        return float(v)

    q = v / s

    if mode == "down":
        q = q.to_integral_value(rounding=ROUND_FLOOR)
    elif mode == "up":
        q = q.to_integral_value(rounding=ROUND_CEILING)
    else:
        q = q.to_integral_value(rounding=ROUND_HALF_UP)

    return float(q * s)
```

Also: if formatted qty is below min qty/notional, **reject**, do not bump to min size and accidentally over-risk.

Pattern 2: tri-state position checks

Do not treat REST failure as "flat."

```
from enum import Enum

class PosState(str, Enum):
    OPEN = "OPEN"
    FLAT = "FLAT"
    UNKNOWN = "UNKNOWN"

def get_position_state(self, symbol: str) -> tuple[PosState, float]:
    res = self._request(
        "GET",
        f"/api/v2/positionRisk",
        params={"symbol": symbol},
        signed=True,
        max_retries=1,
    )

    if res is None:
        return PosState.UNKNOWN, 0.0

    try:
        for p in res:
            if p.get("symbol") == symbol:
                amt = float(p.get("positionAmt", 0.0))
                return (PosState.OPEN if amt != 0.0 else PosState.FLAT), amt
    except Exception:
        return PosState.UNKNOWN, 0.0

    return PosState.UNKNOWN, 0.0
```

In reconciliation:

```
state, amt = broker.get_position_state(symbol)

if state == PosState.UNKNOWN:
    trade["sync_unknown_count"] = trade.get("sync_unknown_count", 0) + 1
    return

if state == PosState.FLAT:
    trade["flat_confirmations"] = trade.get("flat_confirmations", 0) + 1
    if trade["flat_confirmations"] < 2:
        return
    # Now close local trade using linked fills, not last symbol fill.
```

Pattern 3: never use last_fill(symbol) for trade PnL

Store client IDs.

```
entry_client_id = f"E1_{strategy}_{symbol}_{trade_nonce}_ENTRY"
sl_client_id = f"E1_{strategy}_{symbol}_{trade_nonce}_SL"
exit_client_id = f"E1_{strategy}_{symbol}_{trade_nonce}_EXIT"
```

Fetch fills by order IDs or by client prefix and time window.

Your fill ledger should look like:

```
@dataclass
class Fill:
    symbol: str
    order_id: int
    client_order_id: str
    side: str
    price: float
    qty: float
    commission: float
    realized_pnl: float
    maker: bool
    trade_time_ms: int
```

Trade PnL:

```
net_realized = sum(f.realized_pnl for f in fills) - sum(abs(f.commission) for f in fills)
```

Entry and exit VWAP:

```
entry_vwap = sum(f.price * f.qty for f in entry_fills) / sum(f.qty for f in entry_fills)
exit_vwap = sum(f.price * f.qty for f in exit_fills) / sum(f.qty for f in exit_fills)
```

Pattern 4: safe passive timeout with partial fill accounting

If you keep GTX anywhere, handle partials.

```
def _order_exec(order: dict) -> tuple[float, float]:
    qty = float(order.get("executedQty", 0.0) or 0.0)
    quote = float(order.get("cumQuote", 0.0) or 0.0)
    return qty, quote

def wait_cancel_and_account(self, symbol: str, order_id: int, timeout_s: float) -> tuple[float, float, str]:
    deadline = time.time() + timeout_s
    last = {}

    while time.time() < deadline:
        time.sleep(0.1)
        last = self._fetch_order(symbol, order_id) or {}
        status = last.get("status")

        if status == "FILLED":
            qty, quote = _order_exec(last)
            return qty, quote, "FILLED"

        if status in ("CANCELED", "EXPIRED", "REJECTED"):
            qty, quote = _order_exec(last)
            return qty, quote, status

    # Timeout: cancel, then fetch final state because fill/cancel race is possible.
    self._cancel_limit_order(symbol, order_id)
    final = self._fetch_order(symbol, order_id) or last
    qty, quote = _order_exec(final)
    return qty, quote, final.get("status", "UNKNOWN_AFTER_CANCEL")
```

Then only execute the remainder:

```
remaining_qty = target_qty - passive_filled_qty
if remaining_qty > min_qty:
    execute_remainder_loc(...)
```

Pattern 5: marketable limit IOC instead of MARKET fallback

```
def place_ioc_entry(self, symbol: str, side: str, qty: float, bid: float, ask: float,
max_slip_bps: float) -> Optional[dict]:
    if side.upper() == "BUY":
        limit_px = ask * (1.0 + max_slip_bps / 10000.0)
        px = self._format_price(symbol, limit_px, "up")
    else:
        limit_px = bid * (1.0 - max_slip_bps / 10000.0)
        px = self._format_price(symbol, limit_px, "down")

    client_id = f"E1_IOC_{symbol}_{int(time.time_ns() % 1_000_000_000)}"

    params = {
        "symbol": symbol,
        "side": side.upper(),
        "type": "LIMIT",
        "timeInForce": "IOC",
        "quantity": self._format_qty(symbol, qty),
        "price": px,
        "newClientOrderId": client_id,
        "newOrderRespType": "RESULT",
    }

    return self._request("POST", "/fapi/v1/order", params=params, signed=True)
```

This gives you deterministic max slippage. If IOC expires/partially fills, you decide whether to accept partial or abort.

Pattern 6: place protection for filled quantity before marking trade live

```
def place_stop_reduce_only(self, symbol: str, side: str, qty: float, stop_price: float, client_id: str) -> Optional[dict]:
    params = {
        "symbol": symbol,
        "side": side.upper(),
        "type": "STOP_MARKET",
        "stopPrice": str(self._format_price(symbol, stop_price)),
        "quantity": self._format_qty(symbol, qty),
        "reduceOnly": "true",
        "workingType": "MARK_PRICE",
        "newClientOrderId": client_id,
    }
    return self._request("POST", "/fapi/v1/order", params=params, signed=True)
```

Flow:

```
entry filled qty > 0
calculate actual VWAP
calculate actual SL from VWAP
place reduce-only SL for filled qty
verify SL accepted/open
only then state = LIVE
```

If SL placement fails:

```
state = RECOVERY_PENDING
send reduce-only close
wait until exchange confirms flat
```

Pattern 7: release tracker lock during broker I/O

Current `trigger_entry()` holds `self.lock` while calling broker execution. That blocks PnL updates and exit logic.

Better:

```
lock:
    reserve trade_id as PENDING_SUBMIT
unlock

perform broker execution

lock:
    if success:
        update trade to LIVE/PROTECTED
    else:
        remove or mark RECOVERY_PENDING
```

This prevents a slow REST call or GTX timeout from freezing the local trade engine.

Pattern 8: multi-cycle reconciliation confirmation

Do not immediately close local state on one missing REST position.

```
if state == PosState.FLAT:
    trade["flat_seen"] = trade.get("flat_seen", 0) + 1
else:
    trade["flat_seen"] = 0

if trade["flat_seen"] >= 2:
    close_local_from_linked_fills(trade)
```

For safety, require:

```
position flat
and no open bot-owned protective order for that trade
and linked exit fill exists
```

If no linked exit fill exists:

```
state = UNKNOWN
alert
do not fabricate PnL from last symbol fill
```

5. How to model the GTX/taker penalty in backtest

For every live entry, log:

```
signal_ts
decision_mid
decision_bid
decision_ask
spread_bps
book_age_ms
order_sent_ts
ask_ts
first_fill_ts
fill_vwap
maker/taker
fee
qty_requested
qty_filled
post_timeout_ms
fallback_used
mid_after_1s
mid_after_5s
mid_after_30s
```

Define execution cost bps:

For long:

```
entry_cost_bps = (fill_vwap - decision_mid) / decision_mid * 10000 + fee_bps
```

For short:

```
entry_cost_bps = (decision_mid - fill_vwap) / decision_mid * 10000 + fee_bps
```

[Copy](#)

Define maker markout:

For long:

```
markout_5s_bps = (mid_after_5s - fill_vwap) / fill_vwap * 10000
```

[Copy](#)

For short:

```
markout_5s_bps = (fill_vwap - mid_after_5s) / fill_vwap * 10000
```

[Copy](#)

If maker fills have negative 1s/5s markout, your maker strategy is adverse-selected.

Backtest adjustment:

```
if modeled as taker:
    entry = signal_price + side_adjusted_spread/2 + taker_fee + expected_impact

if modeled as passive:
    fill only with calibrated probability
    entry = passive_price
    add conditional adverse markout distribution
    if not filled: either no trade or modeled chase cost
```

[Copy](#)

Do not assume maker spread capture unless live market proves it.



6. Measuring and mitigating STOP_MARKET slippage

For every SL event, log:

```
trade_id
symbol
side
stop_trigger_price
workingType
priceProtect
trigger_event_time
order_update_time
fill_vwap
qty
book_bid_ask_at_trigger
mark_price_at_trigger
contract_price_at_trigger
slippage_bps
```

[Copy](#)

For long stopped out:

```
slippage_bps = (stop_trigger_price - fill_vwap) / stop_trigger_price * 10000
```

[Copy](#)

For short stopped out:

```
slippage_bps = (fill_vwap - stop_trigger_price) / stop_trigger_price * 10000
```

[Copy](#)

Mitigation:



- For protective SL, STOP_MARKET is usually safer than pure stop-limit because stop-limit can fail to fill.
- For very thin symbols, use a marketable stop-limit collar only if you also have an emergency market fallback.
- Increase minimum stop distance by empirical 95th percentile stop slippage.
- Reject trades where:

```
expected_reward < fees + entry_slippage + stop_slippage_buffer
```

[Copy](#)

- Use reduce-only quantity stops and verify they are active.

7. Specific code issues to fix

binance_broker.py

Critical

1. `_cancel_all_account_orders()` on startup can remove protection from existing positions.
 - Reconcile positions first.
 - Cancel only bot-owned stale orders.
 - Never cancel account-wide algos blindly.
2. `_cancel_all_orders(symbol)` before SL placement can remove other strategy stops.
 - Remove this from entry path.
 - Use order IDs/client IDs.
3. `_place_algo_conditional()` likely does not handle Binance errors correctly.
 - `_request()` returns None on HTTP 400, so this code is mostly unreachable:



```
if res and res.get("code") in (-4120, -4138):
```

[Copy](#)

Parse error bodies into structured exceptions.

4. GTX partial fills ignored.
 - Account for PARTIALLY_FILLED.
5. VWAP wrong across slices.
 - Accumulate total_quote and total_qty.
6. MARKET order acceptance does not equal fill.
 - Require confirmed FILLED or user-data fill event.
7. No client ID on GTX orders.
 - Add `newClientOrderId` to all order submissions.
8. Retry of POST without idempotent reconciliation can duplicate orders.
 - On timeout, query by `origClientOrderId` before retry.
9. `_format_qty()` should reject below min qty/notional, not bump to min.

Engine_1.py

Critical

1. Local trade inserted before exchange confirmation.
 - Reserve as pending, then promote to live only after protected fill.
2. Multiple active trades per symbol. 
 - Temporarily block this.
3. reconcile_with_broker() uses order IDs as position IDs.
 - Replace with symbol-level exchange position state.
4. get_last_fill(symbol) corrupts PnL.
 - Replace with linked fill ledger.
5. has_position() returns false on transient REST failure.
 - Use UNKNOWN, not False.
6. Heartbeat bug:

```
getattr(self, 'last_sl_heartbeat', now)
```

should be initialized to 0.0 in __init__ or use default 0.0.

7. Live trailing activation does not match backtest.
 - Use intended_tp_dist or 5 * ATR.
8. Broker result stores exec_sl/exec_tp but does not overwrite local sl/tp.
 - Local exit logic uses stale local values.

Fix after broker success:

```
self.active_trades[trade_id]["entry_price"] = broker_res["exec_entry"]
self.active_trades[trade_id]["sl"] = broker_res["exec_sl"]
self.active_trades[trade_id]["tp"] = broker_res["exec_tp"]
self.active_trades[trade_id]["units"] = broker_res["lot"]
```

9. Broker sync orphan detection references missing method:

```
if hasattr(self.broker, "get_all_positions"):
```

Adapter does not implement it. Orphan detection is effectively dead.

10. Async close callback logs local current_price as exit, not actual fill.
 - Wait for exchange fill event.

six_strategy_engine.py

Important

1. Loss suspension key mismatch.

You set:

```
loss_key = (symbol, direction, strategy)
self.dir_suspend_until[loss_key] = current_bar + 3
```

But check:

```
suspend_key = (symbol, direction)
```

So suspension never triggers.

1. Backtest labels assume exact sim_trade() execution.
 - If live policy changes, labels are stale.
2. Drift detection is good directionally, but it cannot fix execution mismatch.

8. Recommended execution architecture

Target architecture:

```
Signal Engine
|
v
Risk Gate / Portfolio Gate
|
v
Symbol Position Manager
|
v
Execution Router
|
+--> Market Data WS Cache
+--> User Data WS OMS
+--> REST Snapshot/Reconciliation
```

Signal engine responsibilities

Only emits:

```
symbol
direction
strategy
theoretical_entry
theoretical_sl
theoretical_tp/trail
confidence
```

No exchange assumptions.

Risk gate responsibilities

Checks:

```
max one position per symbol
daily risk
max allowed
```

```
portfolio risk
min stop distance
expected execution cost
symbol liquidity
```

Execution router responsibilities

Handles:

```
fresh book
slippage collar
order submit
partial fill
cancel/fill race
idempotent retry
fill wvp
protective stop
```

Copy



OMS responsibilities

Source of truth:

```
user-data WebSocket events
```

Copy

REST is only used for:

```
startup snapshot
reconnect snapshot
periodic audit
```

Copy

Backtester responsibilities

Use calibrated live cost model:

```
entry slippage distribution
stop slippage distribution
fee tier
funding
spread
partial fill probability
latency
min stop floors
exact live trailing policy
```

Copy

9. Priority implementation plan



Today

1. Set max one active trade per symbol.
2. Disable `_cancel_all_orders(symbol)` before new SL.
3. Disable GTX fallback.
4. Use IOC limit with slippage collar.
5. Stop using `get_last_fill(symbol, limit=1)` for PnL.
6. Fix trail activation to 5 ATR.
7. Reject min-stop-floor trades.
8. Store broker `exec_entry`, `exec_sl`, `exec_tp` into local canonical fields.
9. Make `has_position()` tri-state.

Next

1. Add user-data WebSocket.
2. Add book/depth WebSocket cache.
3. Add event-sourced trade state machine.
4. Add fill ledger by `clientOrderid/orderId`.
5. Add protection-first state transitions.
6. Add startup reconciliation that never blindly cancels protective stops.

Then

1. Collect live execution metrics for at least several hundred trades/orders.
2. Fit slippage/markout model.
3. Replay backtest with live execution model.
4. Retrain labels only after the live exit policy is finalized.

Final diagnosis

The live edge is being destroyed less by one isolated and more by a **stacked mismatch**:

```
Backtest:
  independent trades
  exact fills
  exact stops
  SATR trailing
  no exchange state races
  no adverse-selection maker model

Live:
  netted Binance positions
  passive adverse selection
  market chase fallback
  REST-poll state races
  wrong fill attribution
  early trailing
  SL floor R/R distortion
  STOP_MARKET slippage
```

Copy

The first thing to fix is not the ML model. It is the exchange-facing OMS.

Until the OMS is symbol-true, fill-linked, and WebSocket-driven, the backtest/live comparison is contaminated: you cannot tell which losses are strategy losses, execution losses, or bookkeeping artifacts.

...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

...

AI Chat

Stop paying for AI five times →

Ask anything, create anything

+

GPT-5.5 Pro

🗣️ ↩️